Business Logic Design (servidor)

Reglas de negocio del servidor: la capa que valida y persiste las mediciones. Notación: Logical Design & Reverse Engineering Specification v3.

> Esta capa es **100 % independiente del transporte**: solo trabaja con tipos de dominio y con la base de datos descrita en `database_design.md`.

Component Design

1. Tipos de dominio

```
Medicion = (
   id_plaquita: Text,    // nombre que emite la plaquita: "GTI-3A"
   tipo_medicion:  Text,    // "CO2" | "temperatura" | "prueba"
   valor_medicion: Z,       // valor del proyecto (hoy 1234)
   fecha:          Tiempo   // la pone la base de datos (RN3); en la entrada puede
ir vacía
)

FiltroMediciones = (
   desde:          Tiempo,  // opcional
   hasta:          Tiempo,  // opcional
   tipo_medicion:  Text,    // opcional
   valor_medicion: Z        // opcional
)

MedicionesID = { CO2 = 1, TEMPERATURA = 2, PRUEBA = 3 }

ResultadoGuardado = ( ok: B, error: Text | vacío, clase: ClaseError | vacío )
ResultadoLista    = ( ok: B, mediciones: [ Medicion ], error: Text | vacío, clase:
ClaseError | vacío )

ClaseError = { VALIDACION = 'validacion', PERSISTENCIA = 'persistencia' }
```

2. Diagrama del módulo `LogicaDeNegocio`

```
                ------------------- LogicaDeNegocio -------------------
                |
                |
                |
m: Medicion --> guardarMedicion() --> ResultadoGuardado
                |      (valida RN1..RN2; INSERT INTO mediciones; la BD pone la
fecha, RN3)
                |
                |
                |
```

```
                  |
  f: FiltroMediciones --> recuperarMedicion() <-- ResultadoLista
                  |       (SELECT ... ORDER BY fecha DESC LIMIT 50)
                  |
                  ------------------------------------------------------
```

Función	Tipo	Motivo
`guardarMedicion()`	mutante (`-->`)	Escribe en la base de datos (INSERT)
`recuperarMedicion()`	solo lectura (`<--`)	Solo SELECT, no modifica nada

3. Diseño de las funciones (notación v3)

`LogicaDeNegocio.php` no define clases: es un **módulo de funciones**. Las dos públicas (contrato) son `guardarMedicion()` y `recuperarMedicion()`; las demás son **internas** (detalles de implementación, en PHP plano no existen las funciones privadas). A la izquierda, `-->` apunta a la función cuando recibe datos y `<--` sale de ella cuando devuelve.

```
                  ------------------- LogicaDeNegocio -------------------
                  |
                  |
                  |
                  |   conexionBD() --
                  |   validaMedicion( m ) --
                  |   limitesDeFecha( desde, hasta ) --
                  |   resultadoGuardado( ok, error, clase ) --
                  |   resultadoLista( ok, mediciones, error, clase ) --
                  |
                  |
  m: Medicion --> guardarMedicion() --> ResultadoGuardado
                  |
                  |
  f: FiltroMediciones --> recuperarMedicion() <-- ResultadoLista
                  |
                  ------------------------------------------------------
```

Función	Tipo	Entrada	Salida
`guardarMedicion()`	mutante (`-->`)	`m: Medicion`	`ResultadoGuardado`
`recuperarMedicion()`	solo lectura (`<--`)	`f: FiltroMediciones`	`ResultadoLista`
`conexionBD()`	interna (`--x`)	—	`PDO`

Función	Tipo	Entrada	Salida
`validaMedicion()`	interna (`--x`)	`m: Medicion`	`( ok: B, motivo: Text? )`
`limitesDeFecha()`	interna (`--x`)	`desde: Text?, hasta: Text?`	`( desde: Text?, hasta: Text? )`
`resultadoGuardado()`	interna (`--x`)	`ok: B, error: Text?, clase: Text?`	`ResultadoGuardado`
`resultadoLista()`	interna (`--x`)	`ok: B, mediciones: [ Medicion ], error: Text?, clase: Text?`	`ResultadoLista`

4. Reglas de negocio (RN)

Regla	Descripción	Dónde se aplica
RN1	Solo se aceptan tipos de medición conocidos (`"CO2"`, `"temperatura"`, `"prueba"`)	`guardarMedicion()`
RN2	`valor_medicion` es obligatorio y entero	`guardarMedicion()`
RN3	`fecha` la pone la base de datos (`DEFAULT CURRENT_TIMESTAMP`), nunca el cliente	`guardarMedicion()`
RN4	Subir y consultar mediciones son operaciones públicas (sin sesión ni login)	ambas funciones
RN5	Ante un dato inválido o un fallo de la base de datos se devuelve un resultado de error de dominio	ambas funciones

5. Composición interna

```
m: Medicion --> guardarMedicion() --> ResultadoGuardado
                |
                | 1. validar m: RN1 (tipo conocido) y RN2 (valor entero)
                |        ? inválido -> ResultadoGuardado( ok=false,
clase='validacion', error )
                |
                | 2. INSERT INTO mediciones (id_plaquita, tipo_medicion,
valor_medicion)
                |        -> la BD rellena fecha (RN3)
                |
                | 3. -> ResultadoGuardado( ok=true )
                |        (fallo de BD -> ResultadoGuardado( ok=false,
clase='persistencia', error ))

f: FiltroMediciones --> recuperarMedicion() --> ResultadoLista
                |
                | 1. SELECT id_plaquita, tipo_medicion, valor_medicion, fecha
                |        FROM mediciones
```

```
                  |        WHERE <filtros opcionales>
                  |        ORDER BY fecha DESC LIMIT 50
                  |
                  | 2. -> ResultadoLista( ok=true, mediciones )
                  |        (fallo de BD -> ResultadoLista( ok=false,
clase='persistencia', error ))
```

6. Alineación con el diseño de la base de datos

Esta capa es la única que accede a la base de datos, y lo hace ajustándose exactamente a `database_design.md` (tabla `mediciones`):

```
guardarMedicion( m: Medicion )
   = INSERT INTO mediciones (id_plaquita, tipo_medicion, valor_medicion)
     VALUES ( m.id_plaquita, m.tipo_medicion, m.valor_medicion )
     -> ok                                  // sin fecha: la pone la BD (RN3)

recuperarMedicion( f: FiltroMediciones )
   = SELECT id_plaquita, tipo_medicion, valor_medicion, fecha
     FROM mediciones
     WHERE f.desde? / f.hasta? / f.tipo_medicion? / f.valor_medicion?
     ORDER BY fecha DESC LIMIT 50
```

Design Clarifications

- La lógica de negocio tiene **solo dos funciones públicas**: `guardarMedicion()` y `recuperarMedicion()`. Las funciones auxiliares (`conexionBD`, `validaMedicion`, `limitesDeFecha`, `resultadoGuardado`, `resultadoLista`) son detalles de implementación, no forman parte del contrato.
- El **resultado se expresa siempre con tipos de dominio** (`ok`, `mediciones`, `error`, `clase`). No hay `id`: la clave de la fila guardada es la natural compuesta `(id_plaquita, fecha)`.
- `clase` distingue el **tipo de fallo** (`'validacion'` o `'persistencia'`), que es un concepto de dominio puro (datos inválidos frente a fallo al persistir).
- La regla RN2 no fija un valor concreto: `valor_medicion` es el dato que manda el cliente en cada medición.

General Rules

- **Programming Language:** PHP 8 (sin frameworks), acceso a MySQL mediante PDO.
- **Function/Method Headers:** cada función incluye su diseño lógico dentro de un bloque de comentario delimitado por líneas discontinuas (`--------------------`): `entrada --> funcion() --> salida`.
- **Code Readability:** código claro y autoexplicativo; comentarios inline mínimos.
- **Automated Testing:** pruebas unitarias de las funciones críticas (`testLogicaDeNegocio.php`) con un PDO falso, sin necesidad de MySQL.